Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

EscapeManager

Sistema Gestionale per Franchising di Escape Room

Autore:
Adriano Luca Castaldo

N° Matricola:
7138545

Corso principale:
Ingegneria del Software

Docente corso:
Prof. Enrico Vicario

GitHub Repository

Anno Accademico 2025/2026

Indice

1 Introduzione **1**
1.1 Statement . . . 1
1.2 Stack Tecnologico e Strumenti . . . 1
1.3 Note sull'uso dell'Intelligenza Artificiale . . . 2
1.4 Struttura del documento . . . 2
1.5 Stato del prototipo e limitazioni . . . 2

2 Analisi dei Requisiti **3**
2.1 Use Case Diagram . . . 3
2.1.1 Legenda riferimenti a Mockup e Navigazione . . . 4
2.2 Use Case Templates . . . 4
2.2.1 UC1 - Prenotazione Stanza . . . 6
2.2.2 UC1.1 - Annullamento Prenotazione . . . 7
2.2.3 UC2 - Iscrizione Lista d'Attesa . . . 8
2.2.4 UC3 - Aggiornamento Stato Stanza . . . 9
2.2.5 UC4 - Configurazione Tariffe Stanza . . . 10
2.3 Requisiti non funzionali . . . 10
2.4 Diagramma di Navigazione (CLI) . . . 11
2.5 Mockup e Interfaccia Utente (UI) . . . 11

3 Progettazione Architetturale e di Dominio **15**
3.1 Architettura a Strati e BCE . . . 15
3.2 Package Diagram e Dipendenze . . . 16
3.3 Scelte Progettuali e Trade-off . . . 16
3.4 Domain Model (Class Diagram) . . . 17
3.5 Class Diagram dei Controller e Boundary . . . 17
3.6 Class Diagram del Package DAO (Persistenza) . . . 18
3.6.1 Scelte Architetturali e Design Pattern . . . 19
3.6.2 Object Diagram: Istanza Runtime del Pattern Observer . . . 19
3.6.3 State Machine Diagram . . . 20
3.7 Progettazione del Database . . . 20
3.7.1 Diagramma Entity-Relationship . . . 20
3.7.2 Strategie di Mapping e Modello Logico . . . 21
3.8 Dinamica del Sistema (Implementation Perspective) . . . 22
3.8.1 UC1 - Prenotazione Stanza . . . 22
3.8.2 UC4 - Configurazione Tariffe . . . 23
3.8.3 UC3 - Aggiornamento Stato Stanza . . . 24
3.9 Pattern Creazionali: DAO Factory . . . 25

4 Implementazione e Architettura del Codice **27**
4.1 Architettura dei Package . . . 27
4.1.1 cli (Boundary) . . . 27
4.1.2 controllers (Control) . . . 27
4.1.3 domain (Entity) . . . 27
4.1.4 dao (Persistence) . . . 27

4.2 Livello di Persistenza (Pattern DAO) 27
4.3 Configurazione della Connessione al Database 28
4.4 Validazione Input e Gestione Errori 28
4.5 Implementazione dei Design Pattern 28
4.5.1 State Pattern 28
4.5.2 Strategy Pattern 30
4.5.3 Observer Pattern 30
4.5.4 Builder Pattern 30
4.6 Dependency Injection e Testabilità 31
4.7 Gestione Centralizzata delle Eccezioni (Information Hiding) 31
4.8 Gerarchia delle Interfacce e Liskov Substitution Principle 32
4.9 Copia Difensiva (Defensive Copy) 33

5 Collaudo e Testing **34**
5.1 Strategia di Verifica 34
5.2 Setup e Dati di Test 34
5.3 Testing Strutturale (White-Box) 34
5.4 Testing con Mock (Mockito) 35
5.5 Testing di Integrazione (DAO e Database) 36
5.6 Testing Funzionale (Black-Box e Tracciabilità) 36
5.6.1 Catalogo dei Test 36
5.6.2 Tracciabilità Use Case → Test 36

1. Introduzione

1.1 Statement

Il progetto **EscapeManager** è un sistema gestionale pensato per una catena in *franchising* di Escape Room. L'obiettivo è offrire un'unica applicazione che permetta ai clienti di prenotare le stanze e, allo stesso tempo, fornisca al personale gli strumenti necessari per controllare le sessioni di gioco.

Il dominio dell'applicazione si basa su entità principali come **Sede**, **Stanza**, **Utente** (con vari ruoli) e **Prenotazione**. Ogni sede ospita più stanze, ognuna con un tema, una difficoltà, una capienza e un ciclo di vita operativo preciso (es. *Disponibile, In Corso, In Pulizia, In Manutenzione*).

Il sistema prevede tre tipi di utenti:

- **Cliente:** può sfogliare il catalogo delle stanze, prenotare una sessione e mettersi in lista d'attesa per uno slot momentaneamente occupato.
- **Game Master (Operatore):** gestisce le operazioni quotidiane. Controlla le prenotazioni, avvia e termina le sessioni, aggiorna lo stato delle stanze (ad esempio bloccandole per manutenzione) e registra l'esito finale della partita.
- **Admin (Amministratore):** ha una visione globale sul franchising e può configurare le politiche di prezzo (ad esempio impostando i prezzi del weekend).

L'architettura segue il pattern BCE (Boundary-Control-Entity) unito ai principi OOD. Per risolvere i problemi di business (come il cambio di stato delle stanze o il calcolo del prezzo dinamico) ho applicato diversi Design Pattern della Gang of Four. I dati, le prenotazioni e le liste d'attesa sono salvati su un database relazionale.

1.2 Stack Tecnologico e Strumenti

Nello sviluppo sono state utilizzate le seguenti tecnologie:

- **Linguaggio base:** Java (JDK 17).
- **Database:** PostgreSQL. Il codice Java si collega tramite JDBC a un layer DAO per mappare gli oggetti sulle tabelle.
- **Build Tool:** Maven, per gestire le dipendenze e il ciclo di build.
- **Mockup UI:** JavaFX (FXML), usato solo per creare un prototipo visivo slegato dalla business logic.
- **Testing:** JUnit 5 (Jupiter) per i test unitari e di integrazione, e **Mockito** per simulare il database nei Mock test.
- **Versioning e UML:** Git tramite GitHub, IDE Visual Studio Code, e PlantUML integrato per generare i diagrammi direttamente da formato testo.

1.3 Note sull'uso dell'Intelligenza Artificiale

Come previsto dalle direttive del corso, nello sviluppo di *EscapeManager* mi sono avvalso di strumenti di Intelligenza Artificiale Generativa (GitHub Copilot e Google Gemini) come supporto alla programmazione.

L'approccio usato non è stato delegare completamente il lavoro, ma sfruttare l'AI come "aiutante":

- **Decisioni architetturali:** Le scelte di design (uso del pattern BCE, o dove applicare State e Strategy) derivano dallo studio teorico fatto a lezione.
- **Scrittura codice:** Ho chiesto all'AI di aiutarmi ad accelerare la scrittura di codice ripetitivo (ad esempio i metodi getter/setter, l'ossatura delle classi DAO o la base per i test JUnit).
- **Revisione testo e LaTeX:** L'AI è stata utile per impaginare rapidamente i diagrammi e il formattare il testo in LaTeX.
- In ogni caso, ogni frammento di codice o testo suggerito è stato letto, compreso e spesso modificato prima di essere inserito nel progetto.

1.4 Struttura del documento

La relazione è suddivisa in questo modo: il Capitolo 2 presenta l'analisi dei requisiti; il Capitolo 3 discute le scelte architetturali; il Capitolo 4 mostra frammenti di implementazione Java; infine, il Capitolo 5 riassume le batterie di test.

1.5 Stato del prototipo e limitazioni

EscapeManager è attualmente un prototipo scolastico funzionante che si concentra sulla logica di business e sui dati. Per interagirci si utilizza un semplice menu a linea di comando (CLI), mentre le schermate grafiche inserite nella relazione fungono solo da mockup per mostrare il design immaginato.

L'infrastruttura di base (come l'iscrizione a una lista d'attesa modellata con il pattern Observer) è testata e implementata correttamente lato codice. Tuttavia, alcune dinamiche reali (come l'invio concreto di un'email tramite SMTP o un gateway di pagamento) esulano dal perimetro prototipale in esame e non sono state implementate.

2. Analisi dei Requisiti

Per definire i requisiti funzionali del sistema in ottica Object-Oriented, si è proceduto all'identificazione degli attori principali e dei relativi casi d'uso. La modellazione è stata condotta tramite un diagramma dettagliato per l'analisi tecnica.

Il sistema prevede le seguenti figure:

- **Cliente:** Utente finale che interagisce col sistema per consultare il catalogo e prenotare le esperienze.
- **Game Master:** Personale di filiale deputato al controllo operativo delle stanze e delle sessioni.
- **Admin:** Amministratore di sistema incaricato di definire il catalogo e le logiche commerciali.

2.1 Use Case Diagram

La Figura 2.1 rappresenta i casi d'uso effettivi del sistema, organizzati su tre livelli di astrazione:

- **User Goal** (UC1-UC4, UC9-UC10): casi d'uso primari che rappresentano obiettivi completi dell'utente.
- **Function** (UC5-UC7): funzioni riutilizzabili di basso livello, incluse in più casi d'uso tramite la relazione «`include`».
- **Sub-function** (UC1.1): sotto-funzionalità specifiche di un caso d'uso principale.

Il diagramma evidenzia due tipi di relazioni fondamentali:

- **«extend»**: la relazione tra UC2 (Iscrizione Lista d'Attesa) e UC1 (Prenotazione) modella un flusso alternativo opzionale che si attiva solo quando lo slot richiesto risulta occupato.
- **«include»**: UC1 include UC5 (Visualizzazione Catalogo), UC6 (Ricerca Disponibilità) e UC7 (Calcolo Prezzo Dinamico), evidenziando la composizione del processo di prenotazione. UC2 e UC4 includono UC5 per la visualizzazione del catalogo.

Questa struttura rispetta il principio DRY (Don't Repeat Yourself) ed evita la duplicazione della logica di visualizzazione catalogo nei template dei singoli casi d'uso. L'autenticazione utente, pur essendo fondamentale, non è stata modellata come UC separato in quanto rappresenta una precondizione implicita di tutti i casi d'uso e non costituisce un obiettivo utente autonomo.

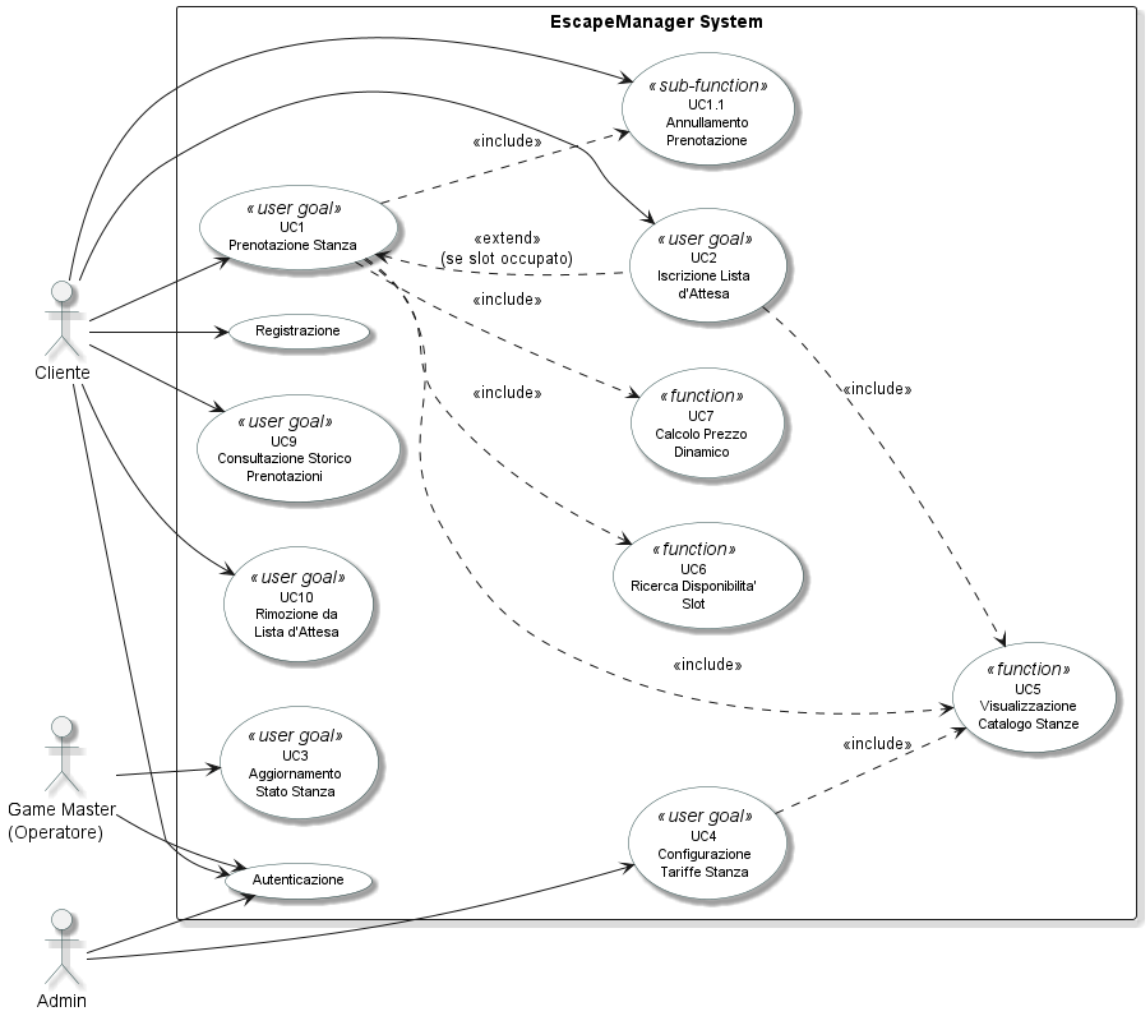

Figura 2.1: Diagramma dei Casi d'Uso

2.1.1 Legenda riferimenti a Mockup e Navigazione

Nei template dei casi d'uso, ogni passo include tra parentesi il riferimento al mockup (**M1..M4**) e al nodo del diagramma di navigazione (**N0..N5**, Figura 2.2).

ID	Mockup associato	Nodo di navigazione
M1	Figura 2.3 (Catalogo e Prenotazione)	N1 Catalogo Stanze, N2 Prenotazione
M2	Figura 2.4 (Lista d'Attesa)	N3 Lista d'Attesa
M3	Figura 2.5 (Stato Stanze)	N4 Aggiorna Stato Stanza
M4	Figura 2.6 (Tariffe)	N5 Configura Tariffe

Tabella 2.1: Mappatura dei riferimenti ai mockup e ai nodi di navigazione

2.2 Use Case Templates

Di seguito vengono dettagliati i template per i casi d'uso primari del sistema, adottando un formato strutturato che esplicita le pre-condizioni, le post-condizioni, il flusso base (Main

Success Scenario) e le possibili deviazioni (Alternative Flows). Ogni passo riporta, tra parentesi, i riferimenti a mockup e navigazione (M1..M4, N0..N5). Questa formalizzazione costituisce il contratto per lo sviluppo e la base per la successiva stesura dei test funzionali.

2.2.1 UC1 - Prenotazione Stanza

Nome Use Case	UC1 - Prenotazione Stanza
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere al cliente di riservare una stanza in uno specifico slot temporale, calcolando la tariffa appropriata.
Pre-condizioni	Il Cliente è autenticato nel sistema. Il sistema ha caricato il catalogo delle stanze.
Post-condizioni	Una nuova Prenotazione viene registrata nel sistema con stato "Confermata". Lo slot orario della stanza risulta occupato.

Flusso Base (Basic Flow)

1. Il Cliente seleziona una Stanza dal catalogo e richiede di visualizzare le disponibilità. (M1, N1)
2. Il Sistema mostra il calendario con le date e gli slot orari disponibili per la Stanza scelta. (M1, N1)
3. Il Cliente seleziona una data, uno slot orario libero e inserisce il numero di giocatori. (M1, N2)
4. Il Sistema calcola il prezzo totale basandosi sulla strategia di prezzo corrente (es. base, weekend) e mostra il riepilogo. (M1, N2)
5. Il Cliente conferma la prenotazione. (M1, N2)
6. Il Sistema registra la prenotazione, aggiorna lo stato dello slot orario e mostra un messaggio di successo. (M1, N2)

Flussi Alternativi (Alternative Flows)

3a. Slot orario non più disponibile (Race condition):

3a.1 Se nel momento della conferma lo slot è stato appena prenotato da un altro utente, il Sistema notifica l'errore.

3a.2 Il Sistema propone al Cliente di tornare al passo 2 o di iscriversi alla Lista d'Attesa (UC2 - *Extend*). (M2, N3)

4a. Numero di giocatori non valido:

4a.1 Se il numero di giocatori inserito supera la capienza massima (o è inferiore alla minima) della Stanza, il Sistema segnala l'errore.

4a.2 Il Cliente è riportato al passo 3 per correggere il dato.

Tabella 2.2: Template del Caso d'Uso: Prenotazione Stanza

2.2.2 UC1.1 - Annullamento Prenotazione

Nome Use Case	UC1.1 - Annullamento Prenotazione
Attore Principale	Cliente
Livello	Sub-function
Scopo	Permettere al cliente di annullare una prenotazione confermata, liberando lo slot per altri utenti.
Pre-condizioni	Il Cliente è autenticato. Esiste almeno una prenotazione "Confermata" associata al Cliente.
Post-condizioni	La prenotazione ha stato "Annullata". Lo slot è nuovamente disponibile. Se la lista d'attesa contiene iscrizioni, il sistema ne notifica uno (Observer Pattern).
Flusso Base (Basic Flow)	
1. Il Cliente visualizza le proprie prenotazioni attive. (M1, N2) 2. Il Cliente seleziona una prenotazione da annullare. (M1, N2) 3. Il Sistema chiede conferma dell'annullamento. (M1, N2) 4. Il Cliente conferma. (M1, N2) 5. Il Sistema aggiorna lo stato della prenotazione a "Annullata" e libera lo slot. (M1, N2) 6. Se la lista d'attesa per la stanza contiene iscrizioni, il sistema notifica il primo observer (UC2 — Observer Pattern). (M2, N3)	
Flussi Alternativi (Alternative Flows)	
3a. Il Cliente rinuncia all'annullamento (M1, N2): 3a.1 Il Cliente seleziona "Annulla" alla richiesta di conferma. 3a.2 Il Sistema interrompe l'operazione.	

Tabella 2.3: Sotto-caso d'uso: Annullamento Prenotazione (UC1.1)

2.2.3 UC2 - Iscrizione Lista d'Attesa

Nome Use Case	UC2 - Iscrizione Lista d'Attesa
Attore Principale	Cliente
Livello	User Goal
Scopo	Permettere a un cliente di iscriversi alla lista d'attesa per una stanza attualmente non disponibile, per essere notificato in caso di liberazione.
Pre-condizioni	Il Cliente è autenticato. Lo slot richiesto per la stanza selezionata non è disponibile.
Post-condizioni	Il Cliente è registrato nella tabella `LISTA_ATTESA` per la stanza selezionata.

Flusso Base (Basic Flow)

1. Il Cliente visualizza che lo slot desiderato è occupato. (M2, N3)
2. Il Cliente seleziona l'opzione "Iscriviti alla Lista d'Attesa". (M2, N3)
3. Il Sistema richiede la conferma per l'iscrizione alla lista d'attesa. (M2, N3)
4. Il Cliente conferma. (M2, N3)
5. Il Sistema registra il Cliente nella lista d'attesa e mostra un messaggio di successo. (M2, N3)

Flussi Alternativi (Alternative Flows)

4a. Il Cliente annulla l'iscrizione (M2, N3):

4a.1 Il Cliente seleziona "Annulla" alla richiesta di conferma.

4a.2 Il Sistema interrompe l'operazione e riporta il Cliente al catalogo.

5a. Cliente già in lista (M2, N3):

5a.1 Il Sistema rileva che il Cliente è già iscritto alla lista d'attesa per quella Stanza.

5a.2 Il Sistema notifica l'utente dell'iscrizione già attiva, senza duplicare la registrazione.

2.2.4 UC3 - Aggiornamento Stato Stanza

Nome Use Case	UC3 - Aggiornamento Stato Stanza
Attore Principale	Game Master
Livello	User Goal
Scopo	Aggiornare il ciclo di vita operativo di una stanza (es. da Disponibile a In Corso, da In Corso a In Pulizia).
Pre-condizioni	Il Game Master è autenticato e autorizzato per la sede di competenza.
Post-condizioni	Lo stato della Stanza è aggiornato nel sistema persistente.

Flusso Base (Basic Flow)

1. Il Game Master accede alla dashboard di controllo delle stanze. (M3, N4)
2. Il Sistema mostra l'elenco delle stanze della sede e il loro stato attuale. (M3, N4)
3. Il Game Master seleziona una Stanza in stato "Disponibile" e clicca su "Avvia Sessione". (M3, N4)
4. Il Sistema valida la transizione di stato. (M3, N4)
5. Il Sistema aggiorna lo stato della stanza in "In Corso" e registra l'evento. (M3, N4)

Flussi Alternativi (Alternative Flows)

4a. Transizione di stato non valida (M3, N4):

4a.1 Il Sistema rileva che l'azione richiesta viola il pattern di stato (es. tentativo di avviare una stanza "In Manutenzione").

4a.2 Il Sistema lancia un'eccezione interna e mostra al Game Master un messaggio di errore: "Transizione non consentita dallo stato attuale".

2.2.5 UC4 - Configurazione Tariffe Stanza

Nome Use Case	UC4 - Configurazione Tariffe Stanza
Attore Principale	Admin
Livello	User Goal
Scopo	Assegnare o modificare la strategia di prezzo applicata a una specifica stanza.
Pre-condizioni	L'Admin è autenticato. La Stanza esiste nel sistema.
Post-condizioni	La nuova strategia di prezzo (PricingStrategy) è associata alla Stanza.
Flusso Base (Basic Flow)	
1. L'Admin accede al pannello di amministrazione e seleziona una Stanza. (M4, N5) 2. Il Sistema mostra i dettagli della stanza e l'attuale strategia di prezzo. (M4, N5) 3. L'Admin seleziona l'opzione per modificare la tariffa e sceglie una nuova strategia dall'elenco (es. *Tariffa Base*, *Tariffa Weekend*). (M4, N5) 4. L'Admin salva la configurazione. (M4, N5) 5. Il Sistema aggiorna i dati nel database e conferma il successo dell'operazione. (M4, N5)	

Altri Casi d'Uso e Funzioni di Supporto Oltre ai casi d'uso principali sopra dettagliati, il sistema implementa diverse funzionalità di supporto e casi d'uso secondari che, pur essendo pienamente funzionali nel codice, presentano una complessità logica minore o agiscono come funzioni di utilità riutilizzabili:

- **UC5 - Visualizzazione Catalogo Stanze («function»):** Espone l'elenco delle stanze filtrabili per sede e tema; è incluso in UC1, UC2 e UC4.
- **UC6 - Ricerca Disponibilità Slot («function»):** Verifica la disponibilità di una stanza interrogando la persistenza; è incluso in UC1.
- **UC7 - Calcolo Prezzo Dinamico («function»):** Applica lo *Strategy Pattern* per il calcolo del preventivo; è incluso in UC1.
- **UC9 - Consultazione Storico Prenotazioni:** Permette al Cliente di visualizzare l'elenco cronologico delle proprie attività passate.
- **UC10 - Rimozione da Lista d'Attesa:** Consente al Cliente di cancellare manualmente una propria iscrizione a una lista d'attesa.

2.3 Requisiti non funzionali

Oltre ai requisiti funzionali, il sistema deve rispettare i seguenti requisiti non funzionali:

- **Prestazioni:** le operazioni principali (ricerca disponibilità, prenotazione, aggiornamento stato) devono completarsi entro 1 secondo su dataset di test (fino a 100 stanze e 1000 prenotazioni).
- **Affidabilità:** le operazioni critiche (es. prenotazione) devono essere atomiche e non lasciare il sistema in stato inconsistente in caso di errore.
- **Usabilità:** la CLI deve guidare l'utente con menu espliciti, validazioni dei dati e messaggi di errore chiari.
- **Manutenibilità:** l'architettura a livelli (CLI, Controllers, DAO, DB) e l'uso di pattern riducono l'accoppiamento e facilitano l'estensione futura.
- **Portabilità:** il sistema deve poter essere eseguito su Windows, macOS e Linux con JDK 17 e PostgreSQL, grazie all'uso di JDBC e configurazioni esterne.

2.4 Diagramma di Navigazione (CLI)

La Figura 2.2 rappresenta il flusso di navigazione della CLI, evidenziando i menu principali e il collegamento diretto con i casi d'uso UC1–UC4. I nodi sono etichettati N0..N5 e sono richiamati nei template dei casi d'uso.

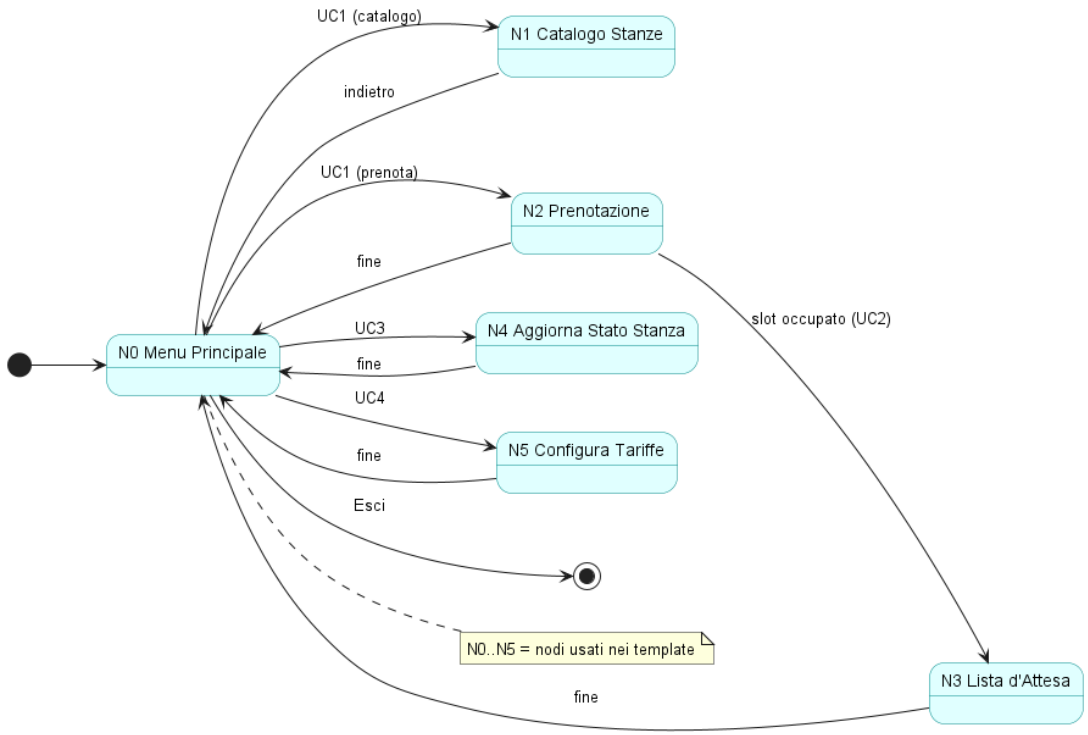

Figura 2.2: Diagramma di navigazione della CLI

2.5 Mockup e Interfaccia Utente (UI)

Al fine di concretizzare l'analisi dei requisiti e validare le interazioni previste nei casi d'uso principali, sono stati realizzati dei mockup per le schermate core del sistema. Questa fase di

prototipazione low-fidelity garantisce una mappatura visiva diretta tra le intenzioni dell'attore (User Goal) e le effettive funzionalità esposte dal sistema *EscapeManager*. I mockup sono identificati come M1..M4 e richiamati nei template.

È importante specificare che, sebbene l'implementazione finale del prototipo utilizzi una **Command Line Interface (CLI)** per focalizzarsi sulla Business Logic, la produzione dei mockup è stata fondamentale in sede di analisi. Essa ha permesso di definire con precisione le interazioni Boundary-Control, garantendo che lo strato di business logic (Application Layer) sia già predisposto e indipendente (secondo il pattern BCE): se in futuro si volesse sviluppare una vera interfaccia web basata su tali mockup, il dominio e i controller rimarrebbero invariati.

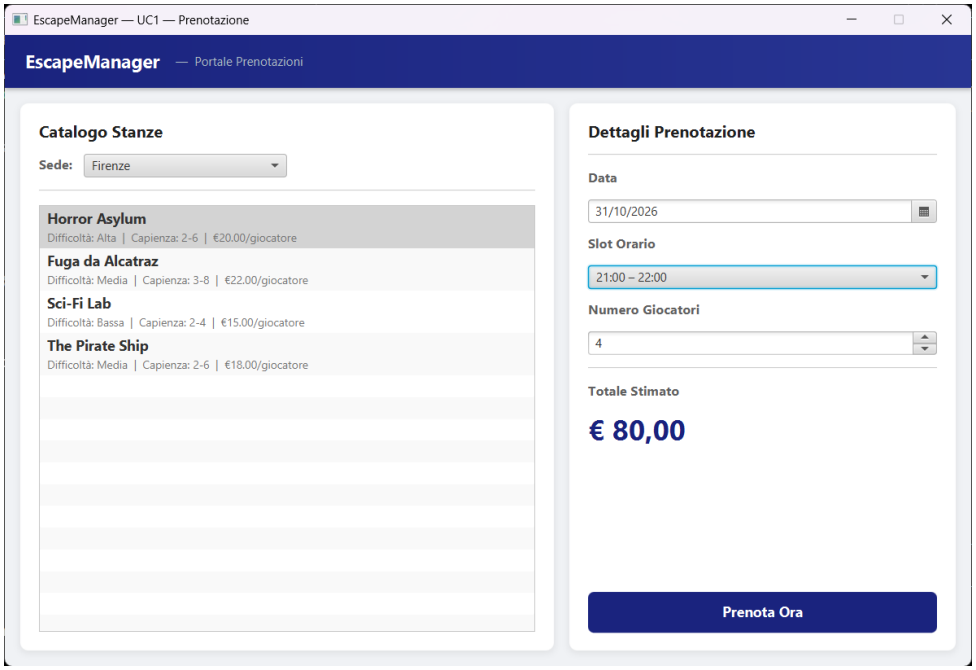

Figura 2.3: [M1] Interfaccia Cliente: Visualizzazione disponibilità e preventivo (UC1)

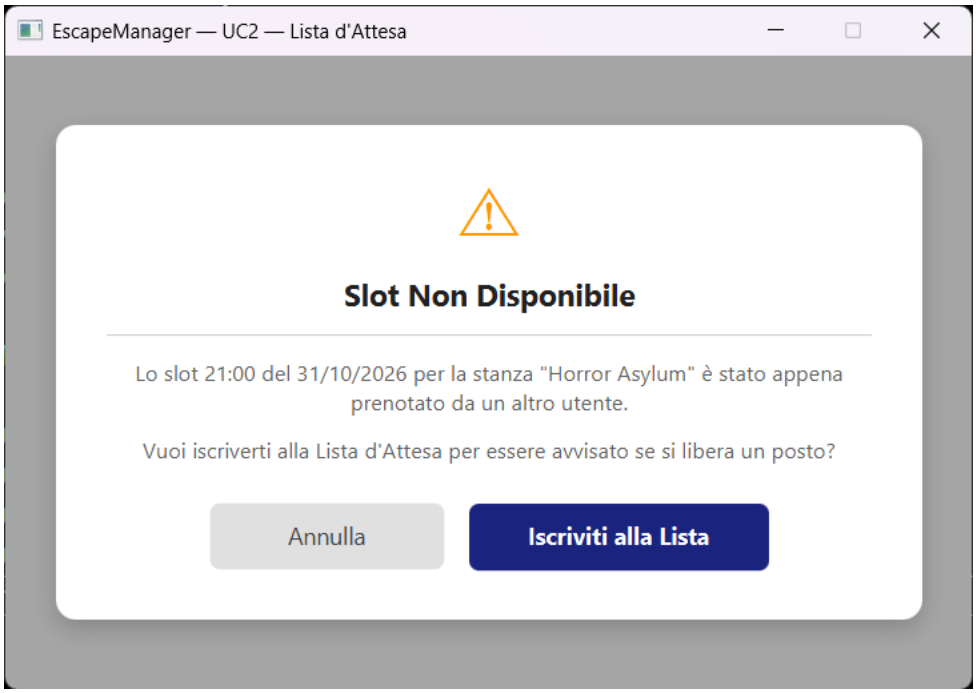

Figura 2.4: [M2] Interfaccia Cliente: Prompt di iscrizione alla Lista d'Attesa (UC2)

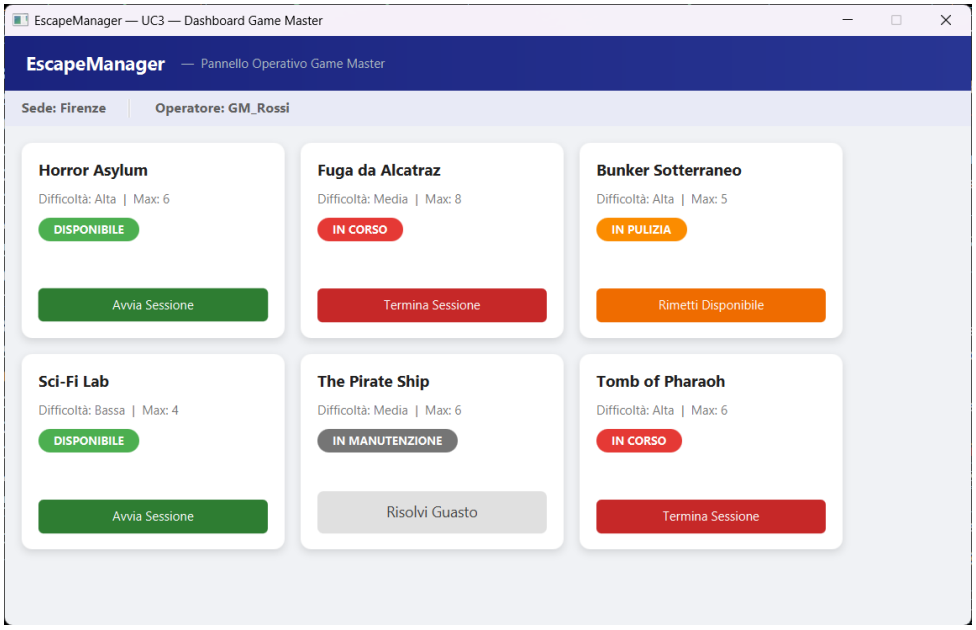

Figura 2.5: [M3] Interfaccia Game Master: Tabellone operativo e controllo stati (UC3)

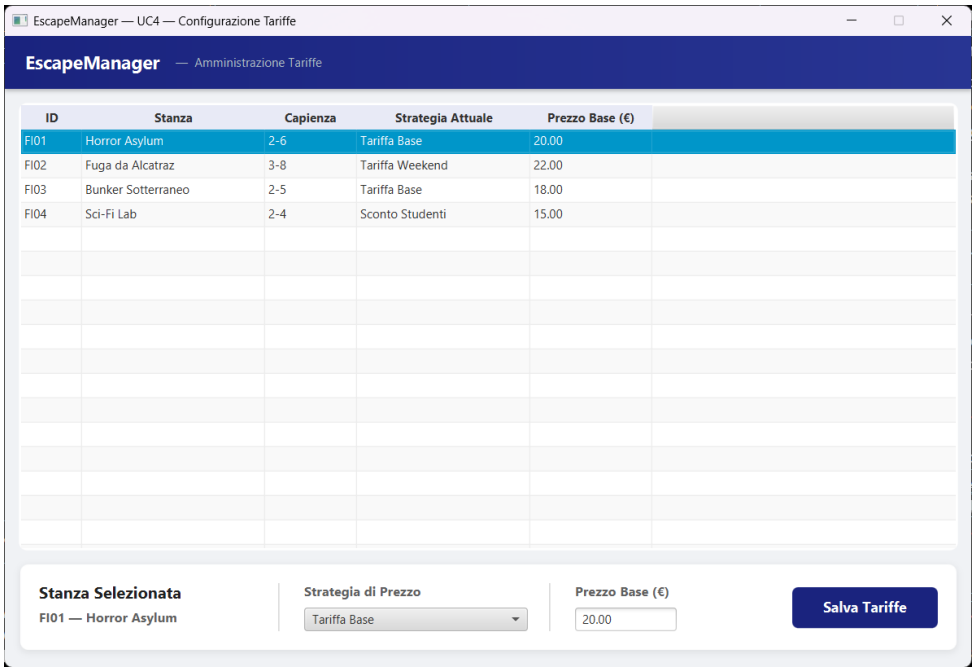

Figura 2.6: [M4] Interfaccia Admin: Configurazione delle strategie di prezzo (UC4)

3. Progettazione Architetturale e di Dominio

Il processo di transizione dall'analisi dei requisiti alla progettazione ha seguito i principi dell'Object-Oriented Design (OOD) e l'approccio BCE (Boundary-Control-Entity). In questo capitolo viene dettagliata la struttura statica del Domain Model, ponendo particolare enfasi sull'applicazione dei Design Pattern fondamentali per garantire la manutenibilità, l'estensibilità e il disaccoppiamento del sistema.

3.1 Architettura a Strati e BCE

L'architettura è organizzata in strati coerenti con il paradigma BCE. Le interazioni tra Boundary (CLI e mockup), Control (Controllers) ed Entity (Domain Model) sono strettamente direzionali, mentre la persistenza è isolata nel layer DAO.

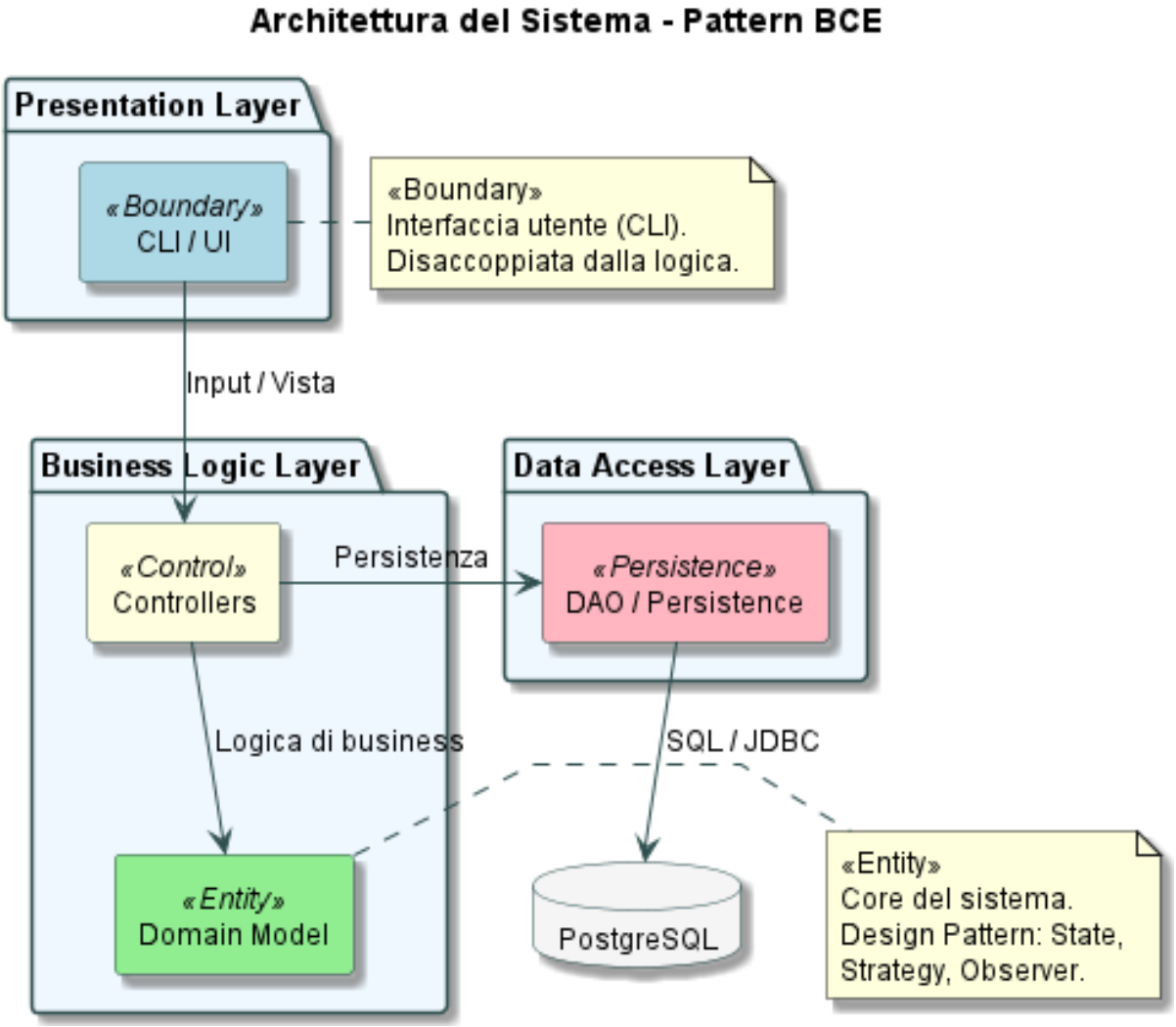

Figura 3.1: Architettura a strati e separazione BCE

3.2 Package Diagram e Dipendenze

Il Package Diagram (Figura 3.2) descrive la struttura logica del progetto e le dipendenze tra i package principali. La CLI accede esclusivamente ai Controllers, i quali orchestrano Domain Model e DAO, mantenendo un flusso di dipendenze unidirezionale.

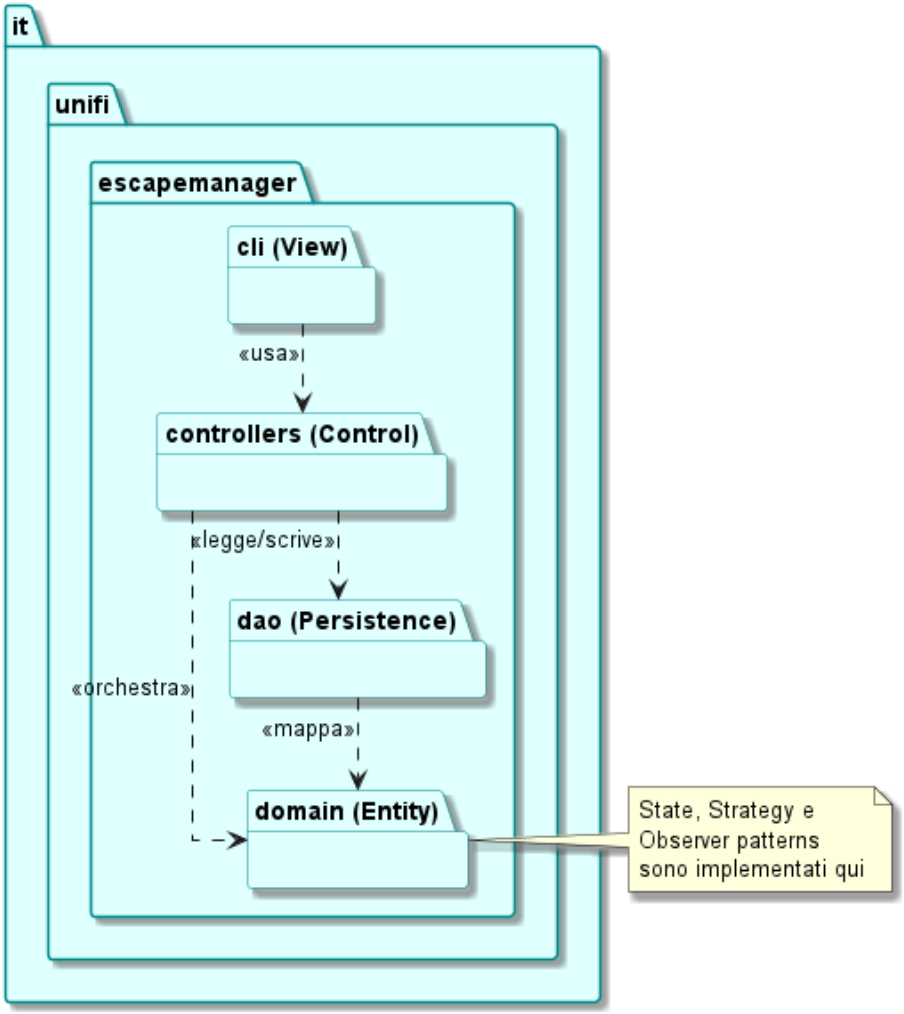

Figura 3.2: Package Diagram dell'architettura di sistema

3.3 Scelte Progettuali e Trade-off

Ecco alcune scelte chiave adottate per mantenere il codice pulito e comprensibile:

- **DAO + JDBC** invece di ORM: architettura leggera studiata per garantire maggiore controllo sul mapping e totale trasparenza dei flussi dati.
- **CLI come Boundary** per ridurre l'impatto della UI e concentrare lo sforzo sulla logica di business.
- **Pattern GoF espliciti** (State/Strategy/Observer) per rendere evidenti i punti di estensione del dominio.

- **Single Table Inheritance** per la gerarchia `Utente`, semplificando query e autenticazione futura.

3.4 Domain Model (Class Diagram)

La Figura 3.3 illustra il diagramma delle classi per il nucleo dell'applicazione (Entity layer). Il modello non include volutamente i dettagli legati all'accesso ai dati (DAO) o all'interfaccia utente (View/CLI), concentrandosi esclusivamente sulle regole del franchise.

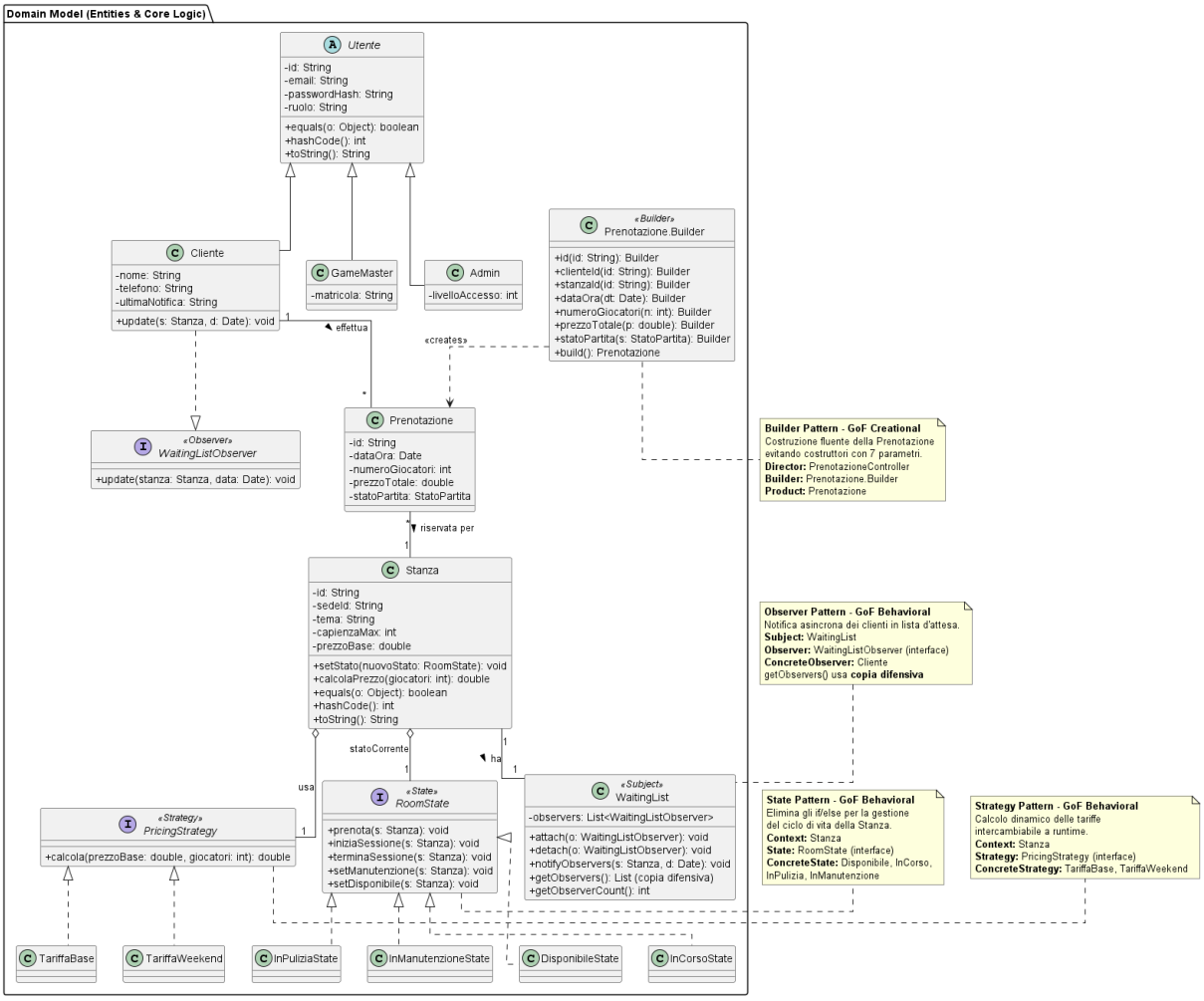

Figura 3.3: Diagramma delle Classi del Domain Model con Design Pattern

3.5 Class Diagram dei Controller e Boundary

Per documentare i layer di controllo e presentazione, la Figura 3.4 mostra invece i Controllers (il livello Control) e la CLI (il livello Boundary).

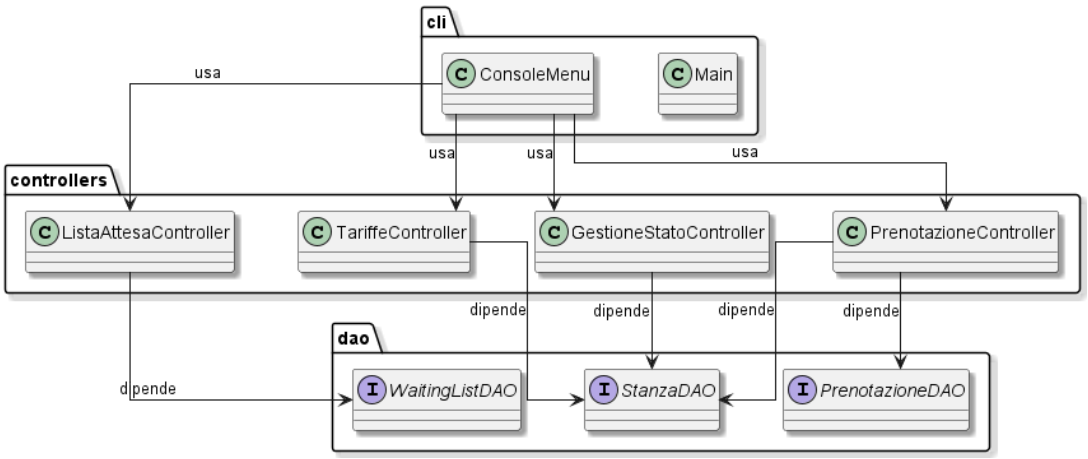

Figura 3.4: Class Diagram: Controllers e Boundary (CLI)

3.6 Class Diagram del Package DAO (Persistenza)

Per non legare i Controller direttamente al database Postgres, il sistema usa il pattern **Abstract Factory** (DAO Factory). La Figura 3.5 mostra il package `dao`, con le interfacce, le classi concrete e la connessione unica (Singleton).

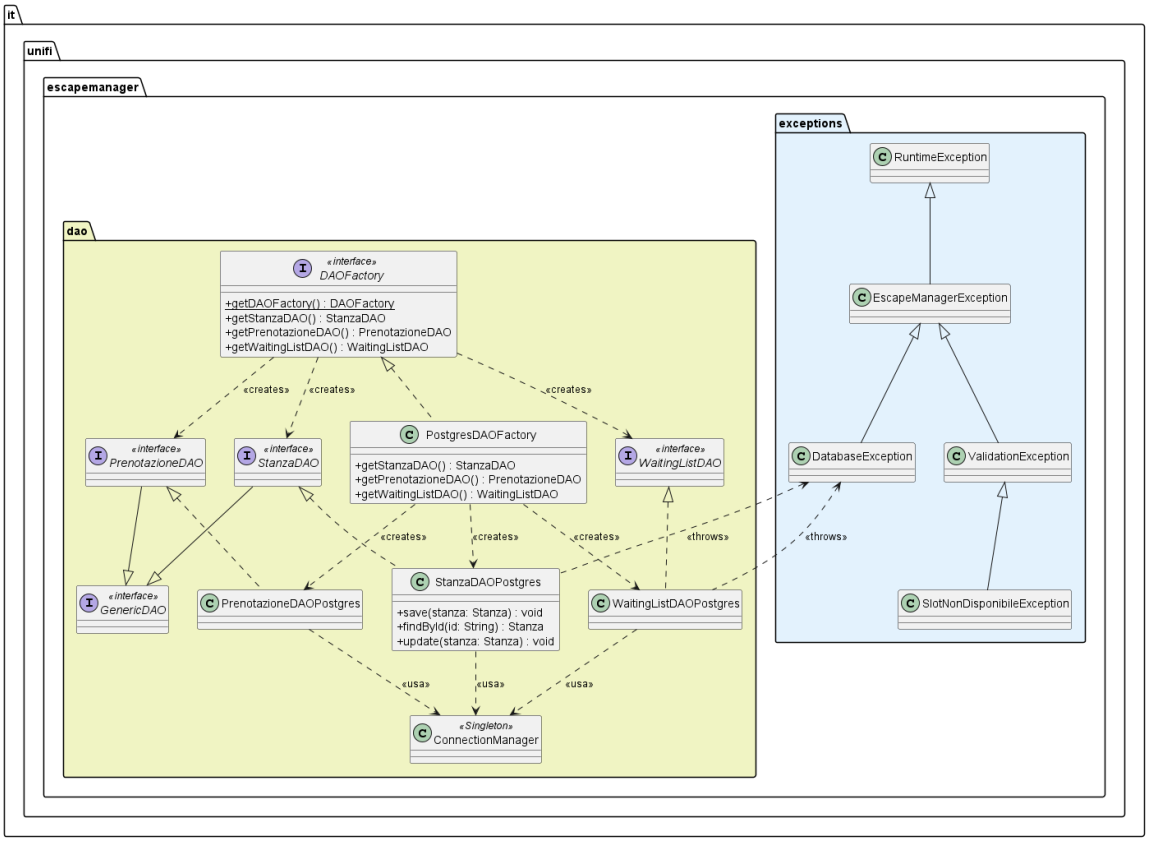

Figura 3.5: Class Diagram: Package DAO e Gerarchia delle eccezioni

3.6.1 Scelte Architetturali e Design Pattern

Per risolvere i problemi principali visti nei casi d'uso (Capitolo 2), ho inserito a livello di dominio tre Design Pattern della *Gang of Four (GoF)*:

- **State Pattern (Gestione ciclo di vita):** Come emerso nell'UC3, la `Stanza` attraversa fasi operative rigide (Disponibile, In Corso, In Pulizia, In Manutenzione). Invece di delegare il controllo a complessi blocchi `switch/case` basati su stringhe o enumerazioni, lo stato è stato reificato nell'interfaccia `RoomState`. Questo garantisce il rispetto del principio *Open-Closed*: l'aggiunta di un nuovo stato futuro non richiederà la modifica del codice della classe `Stanza`.
- **Strategy Pattern (Politiche di Prezzo):** Le regole di tariffazione dipendono da fattori esterni (giorno della settimana, festività, tipologia di clienti). La classe `Stanza` delega il calcolo del costo totale all'interfaccia `PricingStrategy`. L'amministratore (UC4) può così iniettare a runtime strategie diverse (es. `TariffaBase` e `TariffaWeekend`) variando l'algoritmo di calcolo in modo trasparente.
- **Observer Pattern (Lista d'Attesa asincrona):** Per soddisfare il requisito dell'UC2 (Iscrizione Lista d'Attesa), è stato predisposto un meccanismo di Publish-Subscribe. La classe `WaitingList` agisce da *Subject*, mantenendo una lista di `WaitingListObserver` (interfaccia implementata dal `Cliente`). L'infrastruttura di iscrizione è implementata e persiste su database; l'invio automatico delle notifiche è considerato un'estensione futura.

Oltre ai pattern, si sottolinea l'uso dell'ereditarietà per la gestione degli attori (`Utente` come classe astratta generalizzata) e la presenza di un riferimento a `Sede` tramite `sede_id` nel modello dati, che consente di mantenere la coerenza con il concetto di franchising pur non essendo oggetto di operazioni dirette nella CLI.

3.6.2 Object Diagram: Istanza Runtime del Pattern Observer

Per illustrare concretamente il funzionamento del **Pattern Observer** (descritto nel Domain Model), la Figura 3.6 mostra un *Object Diagram* rappresentativo di una specifica configurazione runtime del sistema.

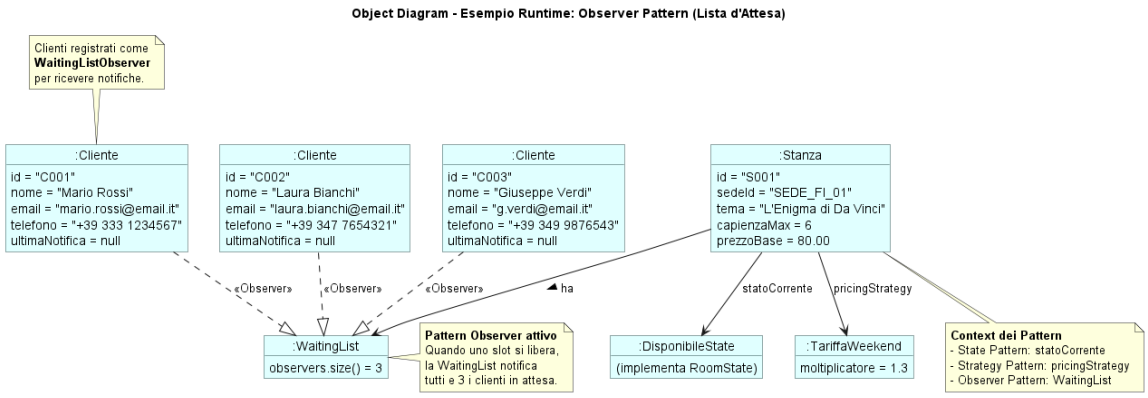

Figura 3.6: Object Diagram: Esempio di istanza runtime del Pattern Observer (Lista d'Attesa)

L'Object Diagram raffigura tre clienti (`:Cliente`) registrati come osservatori (`WaitingList Observer`) sulla `:WaitingList` associata a una stanza specifica (`:Stanza`). Quando lo slot

richiesto si libera, il Subject (`WaitingList`) invocherà il metodo `update()` su tutti e tre gli Observer, notificandoli della disponibilità.

Questo diagramma evidenzia inoltre le relazioni con i pattern **State** (stato corrente della stanza) e **Strategy** (politica di prezzo dinamica), mostrando come i tre pattern collaborino a runtime per realizzare la logica di business del franchising.

3.6.3 State Machine Diagram

Per documentare formalmente il ciclo di vita operativo della Stanza, è stato prodotto il diagramma a stati della Figura 3.7. Questo diagramma è una vista complementare al class diagram del Domain Model e descrive le transizioni valide tra i 4 stati concreti dell'interfaccia `RoomState`.

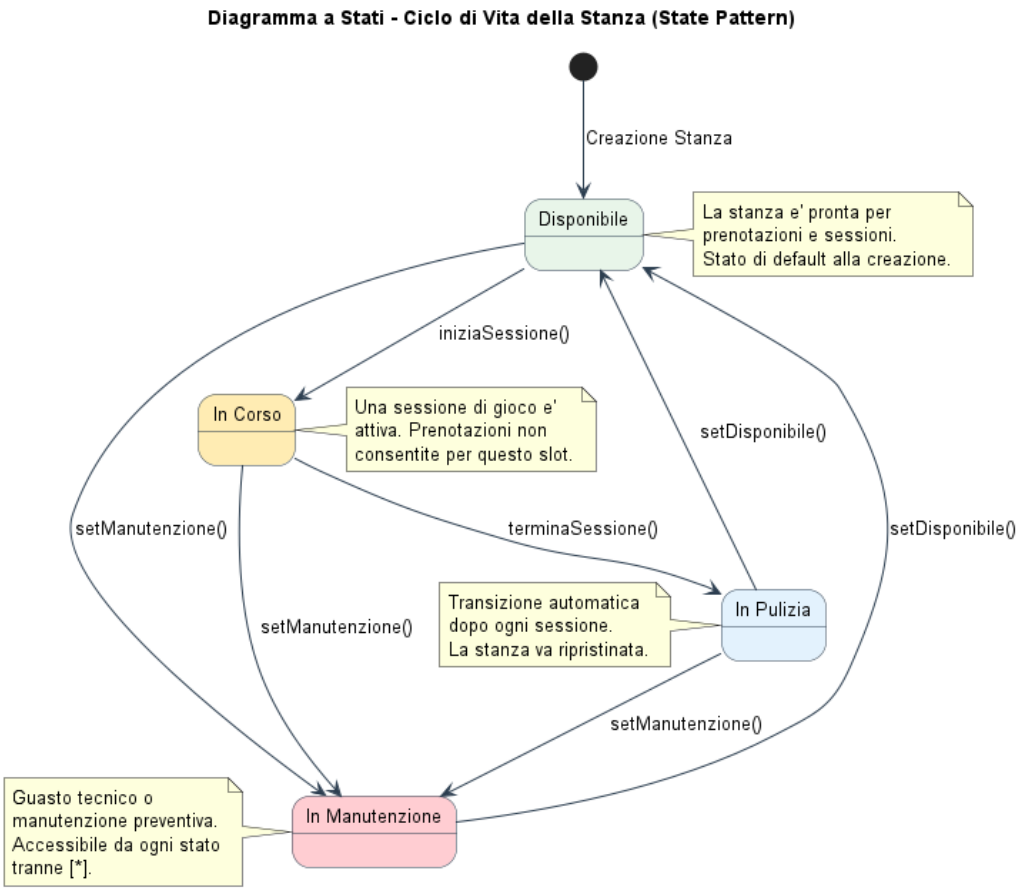

Figura 3.7: State Machine Diagram per il ciclo di vita della Stanza (RoomState)

3.7 Progettazione del Database

Per salvare i dati, il Domain Model a oggetti è stato convertito in un database relazionale (PostgreSQL) usando specifiche strategie di mapping.

3.7.1 Diagramma Entity-Relationship

La Figura 3.8 mostra il diagramma Entità-Relazione (ER) del database, illustrando le tabelle principali, le chiavi primarie (PK) e i vincoli di chiave esterna (FK).

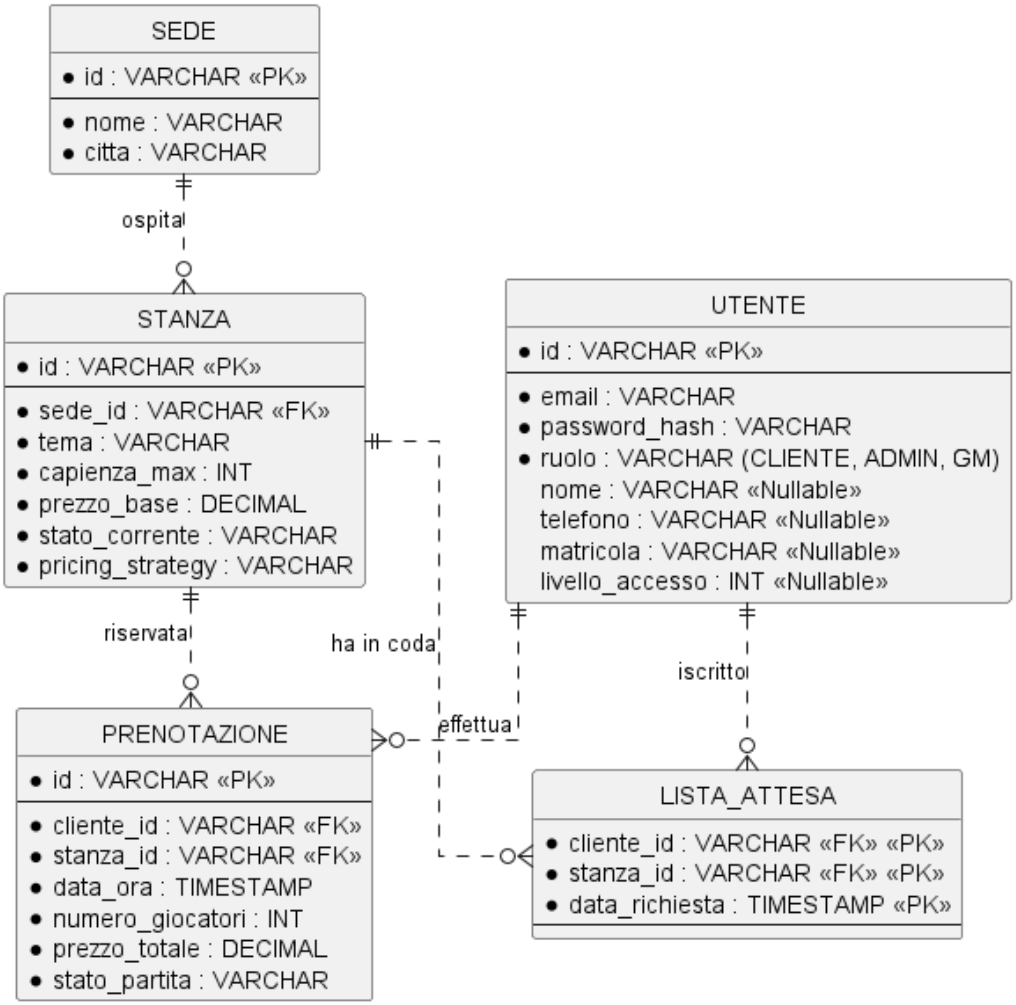

Figura 3.8: Diagramma Entity-Relationship (ER) del sistema

3.7.2 Strategie di Mapping e Modello Logico

Per collegare il mondo a oggetti a quello delle tabelle ho scelto queste strade:

- **Mapping dell'Ereditarietà (Single Table Strategy):** La gerarchia della classe astratta `Utente` (Cliente, GameMaster, Admin) è stata mappata utilizzando il pattern *Single Table Inheritance*. È stata creata un'unica tabella `UTENTE` provvista di una colonna discriminante (`ruolo`). Gli attributi specifici delle sottoclassi (es. `matricola` per il Game Master o `telefono` per il Cliente) sono stati resi *nullable*. Questa scelta ottimizza le query di autenticazione evitando costose operazioni di `JOIN`.

- **Mapping dei Design Pattern (State e Strategy):** Le interfacce `RoomState` e `PricingStrategy` non diventano tabelle separate. Nella tabella `STANZA` sono persistite tramite stringhe (ad es. `stato_corrente=IN_MANUTENZIONE`). A livello di codice Java, i Data Access Object (DAO) utilizzano un meccanismo di *Factory* per istanziare l'oggetto concreto corrispondente al valore letto dal database.

- **Associazioni Molti-a-Molti (Observer Pattern):** La logica Publish-Subscribe della Lista d'Attesa (UC2) si traduce relazionalmente nella tabella di *junction* `LISTA_ATTESA`. Questa

tabella mappa la relazione N:N tra Clienti e Stanze, utilizzando una chiave primaria composita formata dalle foreign key e dal timestamp della richiesta.

Di seguito viene riportato lo **Schema Logico Relazionale** formalizzato:

- **SEDE**(id, nome, citta)
- **STANZA**(id, sede_id, tema, capienza_max, prezzo_base, stato_corrente, pricing_strategy)
- **UTENTE**(id, email, password_hash, ruolo, nome*, telefono*, matricola*, livello_accesso*) — * *attributo nullable a seconda del ruolo*
- **PRENOTAZIONE**(id, cliente_id, stanza_id, data_ora, numero_giocatori, prezzo_totale, stato_partita)
- **LISTA_ATTESA**(cliente_id, stanza_id, data_richiesta)

3.8 Dinamica del Sistema (Implementation Perspective)

Per capire come "parlano" gli oggetti tra loro mentre il programma è in esecuzione, ho disegnato tre Sequence Diagram per i casi d'uso principali.

3.8.1 UC1 - Prenotazione Stanza

La Figura 3.9 mostra il Sequence Diagram relativo al caso d'uso principale. Il diagramma evidenzia il rispetto del pattern architetturale BCE: l'interfaccia utente (Boundary) delega l'elaborazione al `PrenotazioneController` (Control), il quale orchestra il reperimento delle risorse persistenti e demanda la logica matematica di calcolo del prezzo all'entità `Stanza` (Entity). Particolare attenzione è stata posta nel disaccoppiamento del Controller dall'accesso ai dati, delegato interamente a un pattern creazionale.

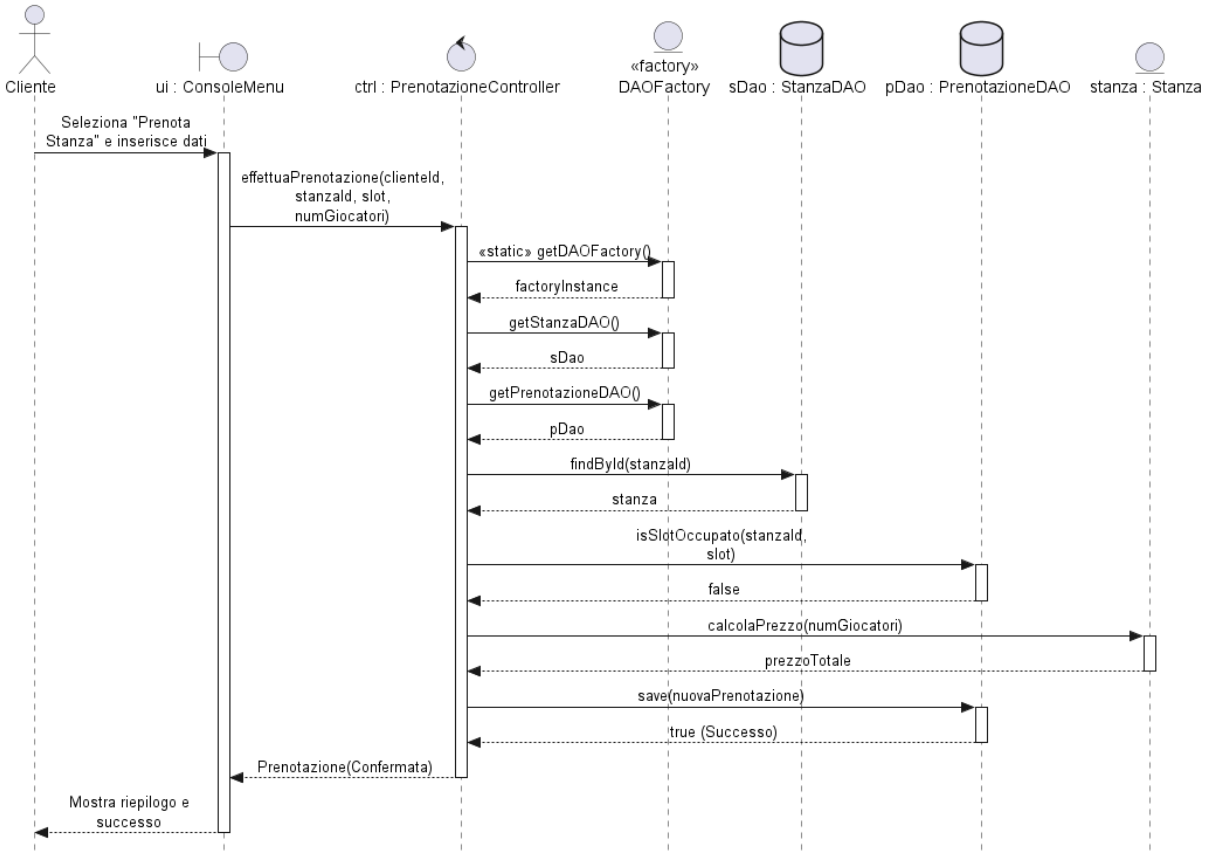

Figura 3.9: Sequence Diagram del caso d'uso UC1 (Prenotazione Stanza)

3.8.2 UC4 - Configurazione Tariffe

La Figura 3.10 descrive il flusso di configurazione delle tariffe. L'Admin opera tramite la CLI, la quale delega al `TariffeController` la selezione della strategia e l'aggiornamento della stanza via DAO.

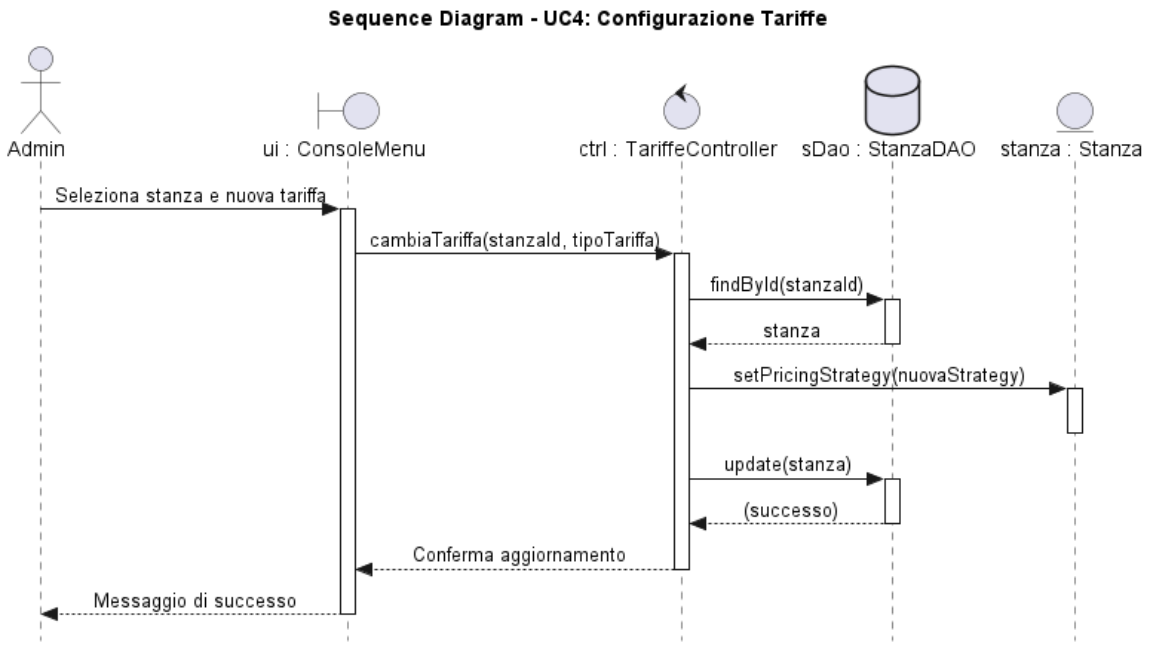

Figura 3.10: Sequence Diagram del caso d'uso UC4 (Configurazione Tariffe)

3.8.3 UC3 - Aggiornamento Stato Stanza

La Figura 3.11 mostra il Sequence Diagram per l'UC3, evidenziando la delega al **State Pattern**. Il Controller non contiene logica di stato: invoca il metodo di transizione sulla `Stanza`, la quale delega l'esecuzione all'oggetto `RoomState` corrente. Il diagramma include anche il flusso alternativo 4a (transizione non valida).

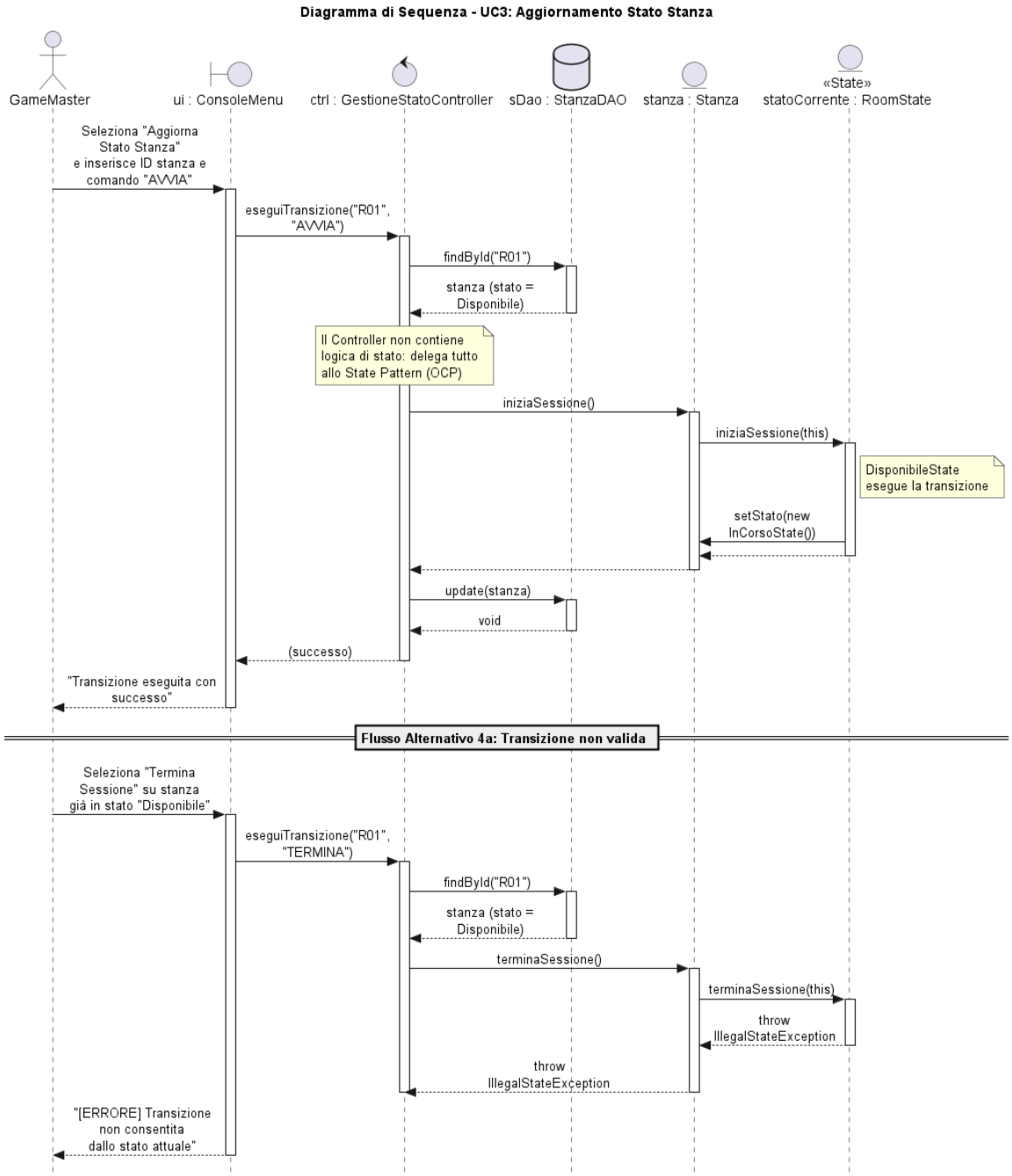

Figura 3.11: Sequence Diagram del caso d'uso UC3 (Aggiornamento Stato Stanza)

3.9 Pattern Creazionali: DAO Factory

Nei diagrammi di sequenza si nota subito che, al momento di salvare i dati, ci si affida a una **DAO Factory** (Abstract Factory). L'idea è scollegare totalmente i Controller dal database PostgreSQL. Infatti, il Controller non crea mai una classe `StanzaDAOPostgres` usando `new`, ma si affida all'interfaccia esposta dalla Factory:

```
// Reperimento dinamico dei DAO indipendentemente dal DB sottostante
DAOFactory factory = DAOFactory.getDAOFactory();
StanzaDAO stanzaDao = factory.getStanzaDAO();
```

```
PrenotazioneDAO prenotazioneDao = factory.getPrenotazioneDAO();
```

Listing 3.1: Utilizzo del pattern DAO Factory nel Controller

Questo vuol dire che se un domani fosse necessario passare a MySQL, basterebbe creare una nuova Factory e i file SQL specifici, senza mai toccare il codice dei Controller o di Dominio.

4. Implementazione e Architettura del Codice

Questo capitolo mostra come il modello di dominio e lo schema del database sono stati trasformati in vero codice Java. Il progetto usa un'architettura "a strati" (Layered architecture) basata sul modello BCE, studiata per separare le responsabilità e semplificare la scrittura dei test.

4.1 Architettura dei Package

L'applicazione è organizzata in quattro package principali (Figura 3.2). Le dipendenze tra i package sono rigorosamente direzionali: lo strato di presentazione (`cli`) non accede mai direttamente al database, ma passa per i `controllers`, che a loro volta orchestrano il `domain` e si interfacciano con i `dao`.

4.1.1 cli (Boundary)

Contiene il punto di ingresso dell'applicazione e la CLI. La classe `ConsoleMenu` gestisce l'interazione con l'utente, la validazione di input di base e la delega delle operazioni ai Controller.

4.1.2 controllers (Control)

Raccoglie i Controller che implementano i flussi dei casi d'uso. Ogni Controller incapsula la logica di coordinamento, chiama i DAO e applica le regole di business esposte dal Domain Model.

4.1.3 domain (Entity)

Include le entità di dominio (`Stanza`, `Prenotazione`, `Utente` e sottoclassi) e l'implementazione dei Design Pattern principali (State, Strategy, Observer).

4.1.4 dao (Persistence)

Contiene le interfacce DAO, le implementazioni PostgreSQL e l'oggetto `ConnectionManager`. L'obiettivo è isolare la persistenza e mantenere il dominio indipendente dalla tecnologia di storage.

4.2 Livello di Persistenza (Pattern DAO)

Per non mischiare la logica dell'applicazione con il codice SQL, l'accesso a PostgreSQL è gestito dal pattern **Data Access Object (DAO)**. In questo modo la business logic non sa (e non deve sapere) nulla di come funziona materialmente il database.

La connessione al database è gestita tramite un **Singleton Pattern** (`ConnectionManager`), che assicura l'esistenza di un'unica istanza di connessione (o di un connection pool) condivisa per tutta l'applicazione, evitando l'overhead di connessioni multiple.

```
public interface GenericDAO<T, ID> {
    void save(T entity);
    T findById(ID id);
```

```
    List<T> findAll();
    void update(T entity);
    void delete(ID id);
}
```

Listing 4.1: Interfaccia generica DAO (`GenericDAO.java`)

```
public interface StanzaDAO extends GenericDAO<Stanza, String> {
    List<Stanza> findBySedeAndStato(String sedeId, String
        statoCorrente);
}
```

Listing 4.2: Specializzazione per il dominio (`StanzaDAO.java`)

In modo analogo, `PrenotazioneDAO` e `WaitingListDAO` incapsulano le operazioni di persistenza per prenotazioni e liste d'attesa.

4.3 Configurazione della Connessione al Database

Le credenziali di accesso al database non sono hardcoded: vengono lette da `db.properties` (in `src/main/resources`) o da variabili d'ambiente, separando configurazione e codice ed evitando la pubblicazione accidentale di segreti.

```
Properties props = loadProperties();
String url = firstNonBlank(
    System.getenv("EM_DB_URL"),
    System.getProperty("db.url"),
    props.getProperty("db.url"),
    "jdbc:postgresql://localhost:5432/escapemanager"
);
```

Listing 4.3: Caricamento della configurazione DB dal classpath

4.4 Validazione Input e Gestione Errori

La CLI valida i formati di input (numeri, date e orari) prima di invocare i Controller. Le eccezioni di business (`ValidationException` e sottoclassi) propagano errori gestibili dalla UI, mantenendo la logica di validazione separata dalla presentazione.

4.5 Implementazione dei Design Pattern

Di seguito vengono riportati i frammenti di codice più significativi che mostrano l'implementazione pratica dei Design Pattern progettati nel Capitolo 3.

4.5.1 State Pattern

Lo State Pattern elimina la necessità di costrutti condizionali complessi per la gestione del ciclo di vita della `Stanza`. Ogni stato concreto implementa il comportamento specifico e gestisce, ove necessario, la transizione allo stato successivo.

```java
public interface RoomState {
    void prenota(Stanza stanza);
    void iniziaSessione(Stanza stanza);
    void terminaSessione(Stanza stanza);
    void setManutenzione(Stanza stanza);
    void setDisponibile(Stanza stanza);
    String getStatusString();
}
```

Listing 4.4: Interfaccia dello State Pattern (`RoomState.java`)

```java
public class DisponibileState implements RoomState {
    @Override
    public void iniziaSessione(Stanza stanza) {
        stanza.setStato(new InCorsoState()); // Transizione di stato
    }

    @Override
    public void terminaSessione(Stanza stanza) {
        throw new IllegalStateException(
            "Impossibile terminare una sessione per una stanza
               DISPONIBILE.");
    }
    // ... altri metodi
}
```

Listing 4.5: Stato concreto (`DisponibileState.java`)

Approfondimento

State Pattern vs Strategy Pattern
Lo **State Pattern** e lo **Strategy Pattern** condividono la stessa struttura meccanica: entrambi delegano il comportamento a un oggetto che implementa un'interfaccia comune, favorendo il polimorfismo rispetto ai costrutti condizionali. Tuttavia, differiscono nell'*intento*:

- **Strategy**: l'algoritmo viene scelto *una tantum* dall'esterno (es. l'Admin seleziona `TariffaWeekend`) e resta fisso per la durata dell'operazione. Le strategie sono intercambiabili ma non si sostituiscono autonomamente.
- **State**: l'oggetto *transiziona automaticamente* tra stati in base alla propria logica interna. Ad esempio, `DisponibileState` sa che dopo `iniziaSessione()` lo stato diventa `InCorsoState`. La transizione è governata dallo stato stesso, non dal client.

Nel progetto, questa distinzione è evidente: la `PricingStrategy` viene impostata esplicitamente dal `TariffeController`. Al contrario, il `RoomState` si auto-sostituisce: ogni stato concreto invoca `stanza.setStato()` all'interno dei propri metodi di transizione, senza intervento esterno.

4.5.2 Strategy Pattern

Il calcolo del prezzo della prenotazione (UC1) è demandato alla `PricingStrategy`. L'Admin, tramite il `TariffeController`, seleziona la strategia appropriata e la imposta sulla stanza.

```java
public interface PricingStrategy {
    double calcola(double prezzoBase, int numeroGiocatori);
}
```

Listing 4.6: Interfaccia dello Strategy Pattern (`PricingStrategy.java`)

```java
public class TariffaWeekend implements PricingStrategy {
    private static final double SOVRAPPREZZO = 1.20; // +20%

    @Override
    public double calcola(double prezzoBase, int numeroGiocatori) {
        return (prezzoBase * numeroGiocatori) * SOVRAPPREZZO;
    }
}
```

Listing 4.7: Strategia concreta (`TariffaWeekend.java`)

4.5.3 Observer Pattern

La lista d'attesa (UC2) implementa un sistema Publisher-Subscriber puro in Java, disaccoppiando l'evento di cancellazione di una prenotazione dall'invio effettivo delle notifiche.

```java
public interface WaitingListObserver {
    void update(Stanza stanza, LocalDateTime slotLiberato);
}
```

Listing 4.8: Interfaccia Observer (`WaitingListObserver.java`)

```java
public class WaitingList {
    private List<WaitingListObserver> observers = new ArrayList<>();

    public void attach(WaitingListObserver observer) {
        observers.add(observer);
    }

    public void notifyObservers(Stanza stanza, LocalDateTime slot) {
        for (WaitingListObserver obs : observers) {
            obs.update(stanza, slot);
        }
    }
}
```

Listing 4.9: Subject concreto (`WaitingList.java`)

4.5.4 Builder Pattern

La classe `Prenotazione` richiede 7 parametri diversi nel costruttore. Per evitare confusione con l'ordine delle variabili ho usato il pattern creazionale **Builder**, che rende il codice molto più leggibile quando si devono inizializzare molti attributi.

```java
Prenotazione p = new Prenotazione.Builder()
        .id(UUID.randomUUID().toString())
        .clienteId("C01")
        .stanzaId("R01")
        .dataOra(LocalDateTime.of(2026, 10, 31, 21, 0))
        .numeroGiocatori(4)
        .prezzoTotale(80.0)
        .statoPartita("CONFERMATA")
        .build();
```

Listing 4.10: Builder Pattern per la classe Prenotazione

4.6 Dependency Injection e Testabilità

I Controller ricevono i DAO tramite costruttore, facilitando la sostituzione delle dipendenze in fase di test. La CLI si limita a comporre l'oggetto e a delegare, mantenendo separati input/output e logica di business.

```java
PrenotazioneDAO prenotazioneDAO = factory.getPrenotazioneDAO();
StanzaDAO stanzaDAO = factory.getStanzaDAO();
PrenotazioneController controller = new PrenotazioneController(
    prenotazioneDAO, stanzaDAO);
```

Listing 4.11: Iniezione delle dipendenze nel controller

4.7 Gestione Centralizzata delle Eccezioni (Information Hiding)

Per il controllo degli errori ho evitato le eccezioni base di Java (come `RuntimeException`), preferendo una gerarchia di eccezioni custom derivata da `EscapeManagerException`.

Come illustrato nella Figura 4.1, la gerarchia divide formalmente gli errori in base al layer architetturale in cui si verificano:

- **`ValidationException`:** utilizzata dal dominio per segnalare violazioni delle invarianti di business (es. la sottoclasse `SlotNonDisponibileException`).
- **`DatabaseException`:** utilizzata dal layer DAO per incapsulare problemi di persistenza.

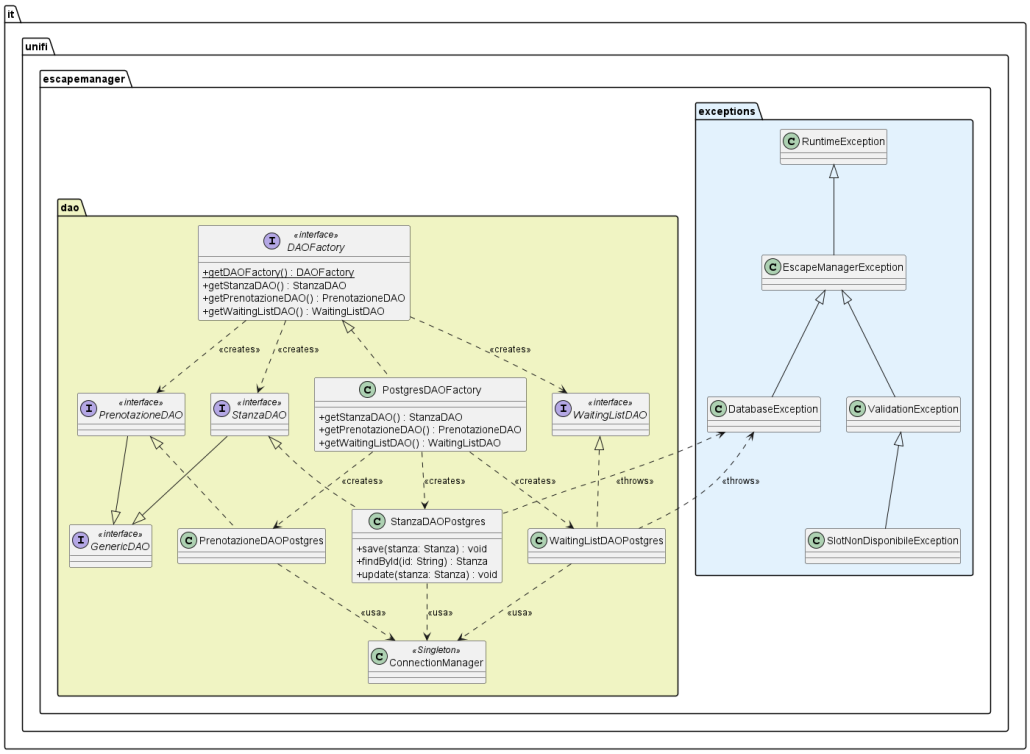

Figura 4.1: Diagramma delle Classi: Pattern DAO Factory e Gerarchia Eccezioni

Questo approccio maschera i dettagli tecnici (Information Hiding): se il driver JDBC lancia una `SQLException`, il livello DAO la cattura e la trasforma in una `DatabaseException`. Così facendo, il Controller che riceve l'errore non deve preoccuparsi di questioni legate a SQL o al driver.

```
try (PreparedStatement stmt = conn.prepareStatement(query)) {
    // Esecuzione query...
} catch (SQLException e) {
    // La SQLException originaria viene nascosta ai livelli superiori
    throw new DatabaseException("Errore di accesso ai dati durante il
        salvataggio.", e);
}
```

Listing 4.12: Incapsulamento delle eccezioni SQL nel layer DAO

4.8 Gerarchia delle Interfacce e Liskov Substitution Principle

Ho creato l'interfaccia `GenericDAO<T, ID>` per avere un contratto di base CRUD per tutte le entità. Le sotto-interfacce come `StanzaDAO` aggiungono le interrogazioni specifiche, e le classi concrete come `StanzaDAOPostgres` contengono le vere query SQL.

Questo design rispetta il **Liskov Substitution Principle (LSP)**: il blocco dei *Controller* comunica solo con l'astrazione (`StanzaDAO`) e ignora totalmente quale sia la classe concreta fornita in quel momento.

```
public interface GenericDAO<T, ID> {
    void save(T entity);
    T findById(ID id);
```

```
    List<T> findAll();
    void update(T entity);
    void delete(ID id);
}

// Specializzazione per il dominio: aggiunge query domain-specific
public interface StanzaDAO extends GenericDAO<Stanza, String> {
    List<Stanza> findBySedeAndStato(String sedeId, String
        statoCorrente);
}
```

Listing 4.13: Gerarchia delle interfacce DAO

4.9 Copia Difensiva (Defensive Copy)

Per tenere al sicuro i dati ho usato una tecnica chiamata **copia difensiva**. La classe `WaitingList` contiene internamente la lista degli iscritti (Observer). Se fornissi la lista tramite un classico getter, chiunque potrebbe manipolarla dall'esterno. Per proteggerla (Information Hiding), `getObservers()` ne restituisce una versione in sola lettura:

```
public List<WaitingListObserver> getObservers() {
    return Collections.unmodifiableList(observers);
}
```

Listing 4.14: Copia difensiva nella WaitingList

Qualsiasi tentativo di aggiungere o rimuovere elementi dalla lista restituita causerà una `UnsupportedOperationException`, proteggendo l'integrità dell'incapsulamento.

5. Collaudo e Testing

Il testing del sistema **EscapeManager** utilizza i framework **JUnit 5** e **Mockito** ed è organizzato su più livelli. Lo scopo, oltre a trovare eventuali bug, è dimostrare che l'architettura funziona come previsto dai requisiti (Capitolo 2).

5.1 Strategia di Verifica

La strategia di test è organizzata su quattro livelli complementari:

- **Unit Test (White-Box):** verificano le logiche pure del Domain Model e dei Design Pattern in completo isolamento.
- **Mock Test (Mockito):** testano i Controller sostituendo i DAO reali con oggetti simulati, isolando la logica di business dal database.
- **Integration Test (DAO/DB):** verificano il mapping ORM e la correttezza delle query principali su database reale.
- **Functional Test (Black-Box):** esercitano i Controller come surrogati della UI, in linea con i template UC.

5.2 Setup e Dati di Test

Per garantire test ripetibili, prima di ogni test di integrazione o funzionale il database viene riportato ad uno stato noto tramite `DatabaseHelper.resetTestDatabase()`, che esegue TRUNCATE e reinserisce dati minimi consistenti (una stanza e due clienti).

5.3 Testing Strutturale (White-Box)

I test strutturali (o Unit Test) si sono concentrati sulle classi del `Domain Model`, con l'obiettivo di massimizzare la copertura (Code Coverage) dei rami condizionali e verificare la correttezza algoritmica, in totale isolamento dalle dipendenze esterne.

Particolare attenzione è stata dedicata al collaudo dei Design Pattern. Il Listing 5.1 mostra un test parametrizzato per la verifica delle politiche di prezzo dinamiche (*Strategy Pattern*).

```
class PricingStrategyTest {

    @Test
    void testTariffaBaseCalcoloCorretto() {
        // Setup
        Stanza stanza = new Stanza("R01", "FI01", "Horror Asylum", 6,
            20.0);
        stanza.setPricingStrategy(new TariffaBase());

        // Execution: 4 giocatori a 20 euro base = 80 euro
        double prezzoCalcolato = stanza.calcolaPrezzo(4);

```

```
        // Assertion
        assertEquals(80.0, prezzoCalcolato);
    }
}
```

Listing 5.1: Unit Test per la logica di calcolo del prezzo (Strategy Pattern)

Oltre al calcolo del prezzo, i test unitari verificano anche le transizioni di stato della `Stanza` (State Pattern, con 11 test per tutte le transizioni valide e illegali), il meccanismo Observer della `WaitingList` (attach, detach, notify), e la costruzione fluente tramite `Builder Pattern`.

5.4 Testing con Mock (Mockito)

Per verificare la correttezza della logica di business dei Controller *indipendentemente* dal database, sono stati implementati test con **Mockito**. Questo framework permette di creare oggetti simulati (mock) delle interfacce DAO, controllando il loro comportamento e verificando le interazioni.

L'annotazione `@ExtendWith(MockitoExtension.class)` abilita l'integrazione automatica con JUnit 5, mentre `@Mock` inietta i DAO simulati e `verify()` asserisce che i metodi attesi siano stati invocati.

```
@ExtendWith(MockitoExtension.class)
class PrenotazioneControllerMockTest {

    @Mock private PrenotazioneDAO prenotazioneDAO;
    @Mock private StanzaDAO stanzaDAO;
    private PrenotazioneController controller;

    @BeforeEach
    void setUp() {
        controller = new PrenotazioneController(prenotazioneDAO,
            stanzaDAO);
    }

    @Test
    void testPrenotazioneRiuscita_FlussoBase_UC1() {
        Stanza stanza = new Stanza("R01", "FI01", "Horror", 6, 20.0);
        when(stanzaDAO.findById("R01")).thenReturn(stanza);
        when(prenotazioneDAO.isSlotOccupato(eq("R01"), any())).
            thenReturn(false);

        Prenotazione result = controller.effettuaPrenotazione("C01",
            "R01",
                LocalDateTime.of(2026, 10, 31, 21, 0), 4);

        assertNotNull(result);
        assertEquals(80.0, result.getPrezzoTotale());
        verify(prenotazioneDAO, times(1)).save(any(Prenotazione.class
            ));
    }
}
```

Listing 5.2: Mock Test: flusso base UC1 con Mockito

5.5 Testing di Integrazione (DAO e Database)

Per verificare il corretto interfacciamento tra l'applicazione Java e il database PostgreSQL (livello ORM), sono stati implementati test di integrazione sulle classi DAO. Per garantire la **ripetibilità** e l'isolamento, il costrutto `@BeforeEach` invoca `DatabaseHelper.resetTestDatabase()`, che esegue TRUNCATE e reinserisce un dataset minimo prima di ogni metodo di test.

```
class StanzaDAOPostgresTest {
    private StanzaDAO stanzaDao;

    @BeforeEach
    void setUp() {
        DatabaseHelper.resetTestDatabase();
        stanzaDao = DAOFactory.getDAOFactory().getStanzaDAO();
    }

    @Test
    void testSaveAndFindById() {
        Stanza stanza = stanzaDao.findById("R01");
        assertNotNull(stanza);
        assertEquals("Horror Asylum", stanza.getTema());
    }
}
```

Listing 5.3: Integration Test per StanzaDAOPostgres con reset dello stato

5.6 Testing Funzionale (Black-Box e Tracciabilità)

Il livello più alto di collaudo ha riguardato i `Controllers`. In questa fase (Black-Box Testing) la logica interna viene ignorata e il sistema viene testato esclusivamente in base agli input e agli output attesi.

Per garantire la massima **tracciabilità**, ogni test funzionale è stato mappato direttamente su uno specifico *Use Case Template* (Sezione 2.2). Sono stati collaudati sia i *Flussi Base* che i *Flussi Alternativi* (le eccezioni).

5.6.1 Catalogo dei Test

La Tabella 5.1 e la Tabella 5.2 riassumono l'intera suite di test, indicando tipo, classe e identificativo.

5.6.2 Tracciabilità Use Case → Test

I test funzionali e con mock sono tracciati direttamente sui template del Capitolo 2. La Tabella 5.3 sintetizza la copertura dei flussi base e alternativi.

ID	Tipo	Test
U1	**Unit**	`StanzaTest.` `testTransizioneCorretta`
U2	**Unit**	`StanzaTest.` `testTransizioneInCorso_TerminaSessione_VaInPulizia`
U3	**Unit**	`StanzaTest.` `testTransizioneDisponibile_ImpostaManutenzione`
U4	**Unit**	`StanzaTest.` `testCicloCompletoStati`
U5	**Unit**	`StanzaTest.` `testEccezioneStatoNonValido (+ 3 varianti)`
U6	**Unit**	`PricingStrategyTest.` `testTariffaBaseCalcoloCorretto`
U7	**Unit**	`PricingStrategyTest.` `testTariffaWeekendMaggiorazione`
U8	**Unit**	`PricingStrategyTest.` `testCambioStrategiaARuntimeDaBaseAWeekend`
U9	**Unit**	`WaitingListObserverTest.` `testAttachENotifica`
U10	**Unit**	`WaitingListObserverTest.` `testDetachNonRiceveNotifica`
U11	**Unit**	`WaitingListObserverTest.` `testNotificaMultipleObservers`
U12	**Unit**	`PrenotazioneBuilderTest.` `testBuilderCreaPrenotazioneCompleta`
M1	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneRiuscita_FlussoBase_UC1`
M2	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneNegata_SlotOccupato_UC1_Alt3a`
M3	**Mock**	`PrenotazioneControllerMockTest.` `testPrenotazioneNegata_CapienzaSuperata_UC1_Alt4a`
M4	**Mock**	`GestioneStatoControllerMockTest.` `testAvviaSessione_DaDisponibile_UC3_FlussoBase`
M5	**Mock**	`GestioneStatoControllerMockTest.` `testTransizioneNonValida_UC3_Alt4a`
M6	**Mock**	`ListaAttesaControllerMockTest.` `testIscrizioneRiuscita_UC2_FlussoBase`
M7	**Mock**	`ListaAttesaControllerMockTest.` `testIscrizioneDuplicata_UC2_Alt5a`
M8	**Mock**	`TariffeControllerMockTest.` `testCambioTariffaWeekend_UC4_FlussoBase`

Tabella 5.1: Catalogo dei test (Parte 1: Unit e Mock Test)

ID	Tipo	Test
I1	**Integration**	`StanzaDAOPostgresTest.testSaveAndFindById`
F1	**Functional**	`PrenotazioneControllerTest.testPrenotazioneNegataPerSlotOccupato_UC1_Alt3a`
F2	**Functional**	`ListaAttesaControllerTest.testIscrizioneListaAttesa_OK`
F3	**Functional**	`ListaAttesaControllerTest.testIscrizioneListaAttesa_Duplicato`
F4	**Functional**	`GestioneStatoControllerTest.testEseguiTransizioneAvvia`
F5	**Functional**	`GestioneStatoControllerTest.testTransizioneNonValida_UC3_Alt4a`
F6	**Functional**	`TariffeControllerTest.testCambioTariffaWeekend_UC4`

Tabella 5.2: Catalogo dei test (Parte 2: Integration e Functional Test)

Use Case / Flusso	Test ID
UC1 - Flusso base (prenotazione)	M1
UC1 - Alternativa 3a (slot occupato)	M2, F1
UC1 - Alternativa 4a (capienza)	M3
UC2 - Flusso base (iscrizione)	M6, F2
UC2 - Alternativa 5a (duplicato)	M7, F3
UC3 - Flusso base (avvio sessione)	M4, F4
UC3 - Alternativa 4a (transizione non valida)	M5, F5
UC4 - Flusso base (cambio tariffa)	M8, F6

Tabella 5.3: Matrice di tracciabilità Use Case → Test